ECOLE POLYTECHNIQUE DE BRUXELLES

2026

GPU Computing

Plane Sweep algorithm implementation

Niccolò Benedetto 626872

Paolo Alberto Bordis 628838

Prof. Daniele Bonatto

PhD Eline Soeten

June 2026

Contents

1 Introduction **1**

2 Computational Resources **2**

3 Experiment **3**

3.1 Explanation 3

3.2 Implementation 4

3.2.1 Naive GPU 4

3.2.2 Naive GPU optimized 5

3.2.3 Gpu shared 6

4 Comparison **7**

5 Conclusion **9**

1 Introduction

Depth estimation is a cornerstone of modern Computer Vision, acting as the essential bridge between flat 2D digital imagery and the volumetric geometry of the real world. By inferring the distance of objects from the observer, this process allows machines to perceive the spatial structure of their environment, which is fundamental for advanced applications such as autonomous navigation, augmented reality, and high-fidelity 3D reconstruction. In this project, we address the computational challenges inherent in this task by implementing the Plane Sweep algorithm on high-performance GPU architectures, aiming to achieve the performance necessary for real-time spatial awareness. The complete project is available online, together with a README.md file that explains how to configure the required variables before running the code:

`https://github.com/dreakess/PlaneSweep505`

2 Computational Resources

Table 2.1: GPU specifications

Device name	NVIDIA GeForce GTX 1080 Ti
Compute capability	6.1
Max Threads per block	1024
Max Threads per Multiprocessor	2048
Shared Memory per block	48 KB
Shared Memory per Multiprocessor	96 KB
Registers per Block/Multiprocessor	65536
Max Grid Size	[2147483647 65535 65535]
Max Block Dimensions	[1024 1024 64]
Single precision FLOP/s (FP32)	3914.99 GFLOP/s
Double precision FLOP/s (FP64)	1957.49 GFLOP/s
#Multiprocessor (SM)	28
Processor Clock Rate	1.607 GHz
Threads per Warp	32
Global Memory Bandwidth	484.44 GB/s
Global Memory Size	10.9999 GB
Constant Memory Size	64 KB
L2 Cache Size	2816 KB
Memory Clock Rate	5505 MHz

3 Experiment

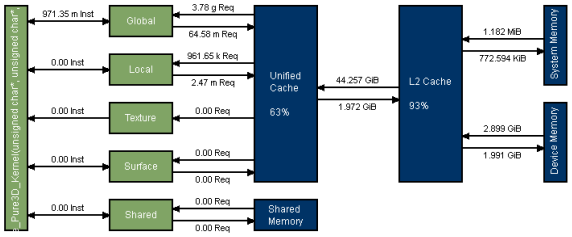
3.1 Explanation

Plane Sweep is an algorithm where the 3D space is discretized into a series of parallel planes. The algorithm performs a sweep across these planes, projecting the reference image onto each plane and comparing it with other available camera views to calculate a cost function, typically based on the Sum of Absolute Differences (SAD). For each pixel, the depth corresponding to the plane that minimizes this cost is selected as the estimated depth. This approach effectively transforms the stereo matching problem into a series of 2D image projections and comparisons, making it inherently well-suited for parallelization on GPU architectures.

The output of this process is a dense 3D structure known as a cost volume (or cost cube). However, the raw cost volume often contains significant noise, as local matching estimates can be weak or ambiguous—particularly in occluded regions. To address this, a regularization step such as Graph Cut is employed. Graph Cut acts as a global optimization technique that enforces spatial consistency, effectively filtering out noise and producing a refined, sharper depth map with smoother transitions between depth discontinuities.

3.2 Implementation

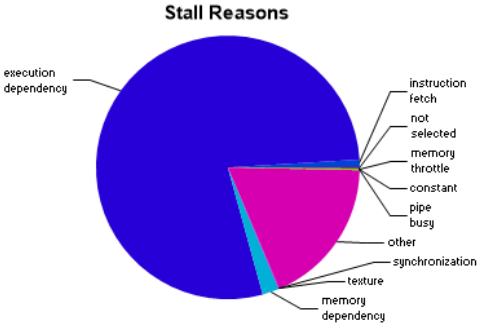
3.2.1 Naive GPU



(a) Memory Statistics

Variable	Achieved	Theoretical	Device Limit	Grid Size: [120,60,256] (2088960 blocks)Block Size: [16,16,1] (256
Occupancy Per SM				
Active Blocks		6	32	
Active Warps	40.63	48	64	
Active Threads		1536	2048	
Occupancy	63.5%	75%	100%	
Warps				
Threads/Block		256	1024	
Warps/Block		8	32	
Block Limit		8	32	
Registers				
Registers/Thread		38	255	
Registers/Block		10240	65536	
Block Limit		6	32	
Shared Memory				
Shared Memory/Block		0	98304	
Block Limit		0	32	

(b) Compute Resources

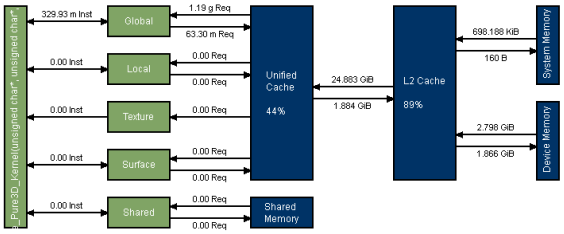


(c) Warp State (Stalls)

Figure 3.1: Detailed analysis using visual profiler for Naive GPU: memory statistics (a), compute Resources (b) and stalls (c).

The implementation of this algorithm is exactly the same as the one used for the CPU version. For the general kernel setup, a block size of [16,16] is used, while the grid size is [120,60,256], corresponding to the dimensions of the cost cube. Obviously, the kernel is launched once for each camera, and the core of the SAD computation is performed inside the kernel. From the analysis in the figure 3.1, we can see that the actived warps spent most of their time in execution. In this implementation, without any optimization, there are also more memory transfers between the L2 cache and device memory. This implementation clearly outperforms the naive CPU version (see 4.1), but with further optimizations we can achieve even better performance.

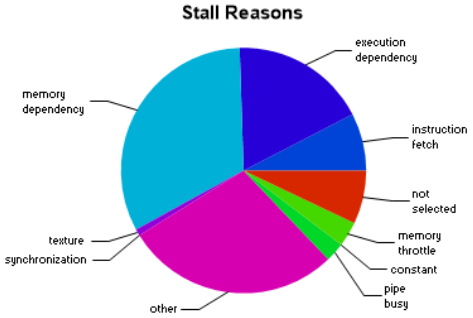
3.2.2 Naive GPU optimized



(a) Memory Statistics

Variable	Achieved	Theoretical	Device Limit
Occupancy Per SM			
Active Blocks		8	32
Active Warps	57.36	64	64
Active Threads		2048	2048
Occupancy	89.6%	100%	100%
Warps			
Threads/Block		256	1024
Warps/Block		8	32
Block Limit		8	32
Registers			
Registers/Thread		31	255
Registers/Block		8192	65536
Block Limit		8	32
Shared Memory			
Shared Memory/Block		0	98304
Block Limit		0	32

(b) Compute Resources

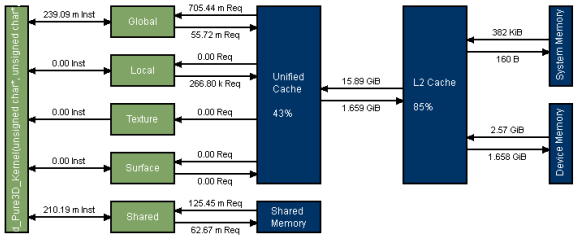


(c) Warp State (Stalls)

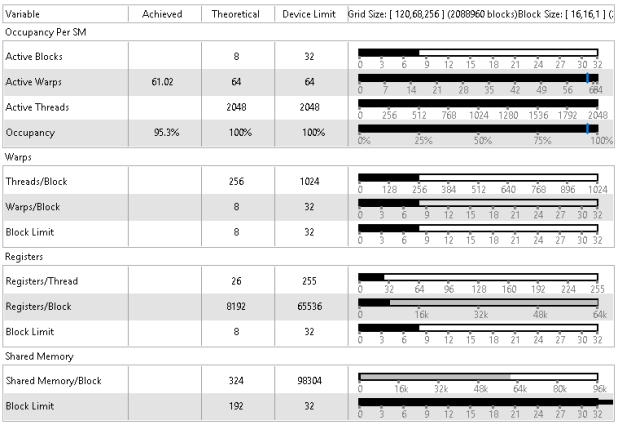
Figure 3.2: Detailed analysis using visual profiler for Opt Naive GPU: memory statistics (a), compute Resources (b) and stalls (c).

In this setup, all variables use the float type instead of double, since CUDA is optimized for this data type. Another optimization consists of using constant memory instead of device memory, because the camera parameters do not change during the computation. This allows us to exploit this memory space to improve loading performance. As we can see, these optimizations increase occupancy and reduce the time spent on execution dependencies. They also reduce the pressure on the L2 cache, which has to handle fewer load and store operations to device memory.

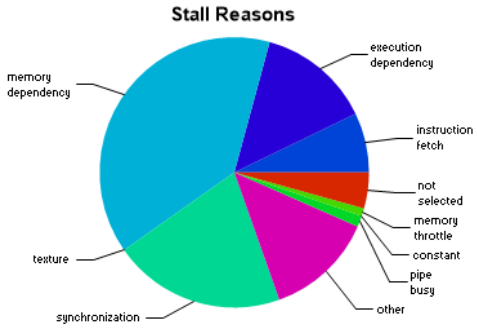
3.2.3 Gpu shared



(a) Memory Statistics



(b) Compute Resources



(c) Warp State (Stalls)

Figure 3.3: Detailed analysis using visual profiler for SHARED GPU: memory statistics (a), compute Resources (b) and stalls (c).

This implementation is different from the previous two. In terms of execution time, it still shows similar performance (see 4.1), as will be analyzed in the next chapter, but it differs in other aspects. In practice, we implemented shared memory by loading the entire reference camera. With this approach, we reduced the number of accesses to the reference camera information. For this reason, the warp state shows more synchronization than in the previous experiment.

4 Comparison

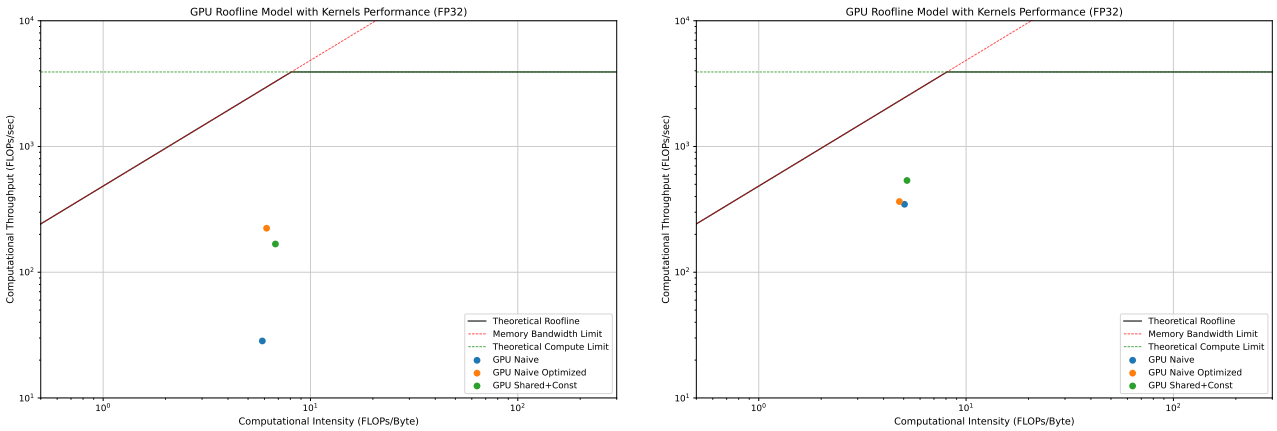
Algorithm & Strategy	**Kernel Time** [s]	**Speedup** [x]	**Performance** [GFLOP/s]
Naive CPU	149.069	1.00x	0.53
Naive GPU	0.9904	151.00x	28.4
Naive Optimized GPU	0.1280	1164.00x	223.85
Shared	0.1709	872.00x	167.72

Table 4.1: Performance comparison of the Plane Sweeping depth estimation algorithm across different optimization stages. Benchmarks were recorded at a resolution of 1920×1080 with 256 depth planes ($RAD = 1$). The Speedup is calculated relative to the unoptimized Naive Baseline based on Kernel Time.

The naive optimized GPU version achieves the best results because the GTX 1080 Ti effectively uses the L2 cache to load data quickly.

The shared-memory version is faster than the CPU baseline, but it does not outperform the naive optimized one. This happens because the synchronization overhead required by the shared memory cancels out the benefits of reducing global memory accesses.

To better understand this behavior, we evaluated the roofline model. Moreover, to better explain the impact of shared memory, we tested a different scenario by increasing the RAD value to 11. This allows us to analyze the relationship between the workload and the computational capacity.



(a) $rad = 1$. (b) $rad = 11$

Figure 4.1: Comparison between two different rad

As illustrated in the Roofline model for an increased workload 4.1, the higher computational intensity changes the architectural bottleneck . In this scenario, the Shared Memory implementation successfully amortizes its thread synchronization overhead, outperforming the Naive Optimized GPU version as the kernel becomes compute-bound rather than limited by memory access latency .

5 Conclusion

The current results (see 4.1) show that, for the plane sweep algorithm, an implementation based on shared memory is preferable when RAD is equal to 11, whereas for smaller RAD values, such as RAD = 1, the optimized GPU implementation is preferable. Using a GTX 1080 Ti highlights these architectural trade-offs, as its characteristics allow the naive optimized version to excel at lower intensities without the synchronization constraints of manual memory management. To better evaluate this type of hardware under higher stress, future benchmarks should increase the image resolution to 2K or 4K and utilize more than two secondary camera views in addition to the reference camera.